

RaktaSetu – Blood Bank Management System

A Project Report Submitted in the partial fulfillment of the requirements for the award of the degree of

BACHELOR OF TECHNOLOGY

In

DEPARTMENT OF COMPUTER SCIENCE ENGINEERING

&

DEPARTMENT OF COMPUTER SCIENCE AND INFORMATION TECHNOLOGY

By

KAIRAV VASHI

2520030582

P. SAI HARSHITH REDDY

2520030375

PAKALAPATI HARINI

2520030177

Under the Esteemed Guidance of

Dr. Spandana S.

Assistant Professor

Department of Computer Science and Engineering



Koneru Lakshmaiah Education Foundation

(Deemed to be University estd. u/s. 3 of the UGC Act, 1956)

Off-Campus: Bachupally-Gandimaisamma Road, Bowrampet, Hyderabad, Telangana - 500 043.

Phone No: 7815926816, www.klh.edu.in

**K L (Deemed to be) University
DEPARTMENT OF COMPUTER SCIENCE ENGINEERING**

&

DEPARTMENT OF COMPUTER SCIENCE AND INFORMATION TECHNOLOGY



Declaration

The Project Report entitled “RaktaSetu – Blood Bank Management System” is a record of Bonafide work of KAIRAV VASHI (2520030582), P. SAI HARSHITH REDDY (2520030375) and PAKALAPATI HARINI (2520030177) submitted in partial fulfillment for the award of B. Tech in Computer Science and Engineering (or) Computer Science and Information Technology to the K L University.

The results embodied in this report are based on the development and implementation of the RaktaSetu blood bank management application and have not been copied from any other department, university, or institute.

KAIRAV VASHI – 2520030582

P. SAI HARSHITH REDDY – 2520030375

PAKALAPATI HARINI – 2520030177

K L (Deemed to be) University

DEPARTMENT OF COMPUTER SCIENCE ENGINEERING

&

DEPARTMENT OF COMPUTER SCIENCE AND INFORMATION TECHNOLOGY



CERTIFICATE

This is certify that the mini project based report entitled “RaktaSetu – Blood Bank Management System” is a bonafide work done and submitted by KAIRAV VASHI (2520030582), P. SAI HARSHITH REDDY (2520030375) and PAKALAPATI HARINI (2520030177) in partial fulfillment of the requirements for the award of the degree of BACHELOR OF TECHNOLOGY in Department of Computer Science Engineering, K L (Deemed to be) University.

The project was developed as a Database Management System application during the academic year 20206-27.

Signature of the Guide

Signature of the Course Coordinator

Signature of the HOD

ACKNOWLEDGEMENT

We sincerely thank our guide, course coordinator, faculty members, and the Department of Computer Science and Engineering for their guidance and support during the development of RaktaSetu.

We are grateful for the academic guidance and feedback that helped us understand database design, relational modelling, REST API development, authentication, and full-stack integration.

We also thank the teaching and non-teaching staff for providing the facilities and environment required to design, implement, test, and demonstrate the project.

Finally, we thank our parents, friends, and classmates for their encouragement and cooperation throughout the project.

KAIRAV VASHI – 2520030582

P. SAI HARSHITH REDDY – 2520030375

PAKALAPATI HARINI – 2520030177

Abstract

RaktaSetu is a database-driven Blood Bank Management System designed to connect donors, hospitals, and administrators through a unified web application. The system manages donor profiles, blood donations, blood inventory, hospital blood requests, donation camps, cross-match records, alerts, and administrative reports. Its main goal is to reduce manual coordination and provide a structured workflow for managing blood availability and requests.

The application provides separate portals for donors, hospitals, and administrators. Donors can maintain their profile, view donation history, see nearby blood requests, and receive match-related alerts. Hospitals can search inventory, raise requests, track their requests, and view request details. Administrators manage inventory batches, requests, donation records, camps, cross-matching, alerts, and dashboard analytics.

The backend uses Node.js with Express and MySQL. The database uses normalized relational tables and an inventory view, while transactions are used for operations such as recording donations and fulfilling requests. JWT-based authentication and role-based authorization separate donor, hospital, and admin access. The frontend is implemented with React and Vite and communicates with the backend through REST APIs using JSON.

Index

S. No.	Chapters	Topics	Page.no
		Acknowledgement	
		Abstract	
1	Introduction	1.1 Background of the project 1.2 Problem statement 1.3 Scope of the project 1.4 Objectives 1.5 Importance of the application 1.6 Target users/audience	5–7
2	System Requirements	2.1 Hardware requirements 2.2 Software requirements 2.3 Development tools and frameworks	8–9
3	Technology Stack	3.1 Front-end 3.2 Back-end 3.3 Database 3.4 Version control 3.5 APIs / external services	10–12
4	System Architecture	4.1 High-level architecture 4.2 Frontend, backend and database layers 4.3 Local deployment architecture	13–14
5	Design	5.1 Database design overview 5.2 ER diagram / database schema	15–16
6	Implementation	6.1 Module-wise implementation 6.2 Front-end logic 6.3 API integration 6.4 Authentication & authorization	17–20
7	Features	7.1 Main features 7.2 How each feature works	21–24
8	Testing	8.1 API testing 8.2 Integration testing 8.3 Testing tools 8.4 Test cases and results	25–27
9	Deployment	9.1 Steps to deploy 9.2 Environment configuration 9.3 Frontend/backend hosting	28–29
10	Challenges & Limitations	10.1 Issues faced 10.2 Solutions applied 10.3 Current limitations	30–31
11	Future Enhancements	11.1 Planned features 11.2 Possible integrations and optimisations	32
12	Conclusion	12.1 Summary 12.2 What was achieved 12.3 Skills learned	33
13	References	Books, tutorials, APIs, tools and documentation	34

14	Appendices	Screenshots of the app Sample code snippets Installation/setup instructions User manual or guide	35–38
----	------------	---	-------

CHAPTER -1 INTRODUCTION

1.1 Background of the Project

Blood banks coordinate a time-sensitive resource in which availability depends on donations, blood groups, storage limits, hospital demand, and correct allocation. When these activities are maintained manually or across disconnected records, it becomes difficult to obtain a single, current view of available blood and outstanding requests.

RaktaSetu was developed as a web-based Blood Bank Management System to organize these activities in one database-driven application. The system connects three principal roles: donors, hospitals, and administrators. Donor and hospital profiles are stored separately from login credentials, while operational data such as donations, requests, inventory batches, camps, cross-match records, and alerts are maintained in related tables.

The project demonstrates relational schema design, primary and foreign keys, constraints, indexes, SQL queries, views, transactions, and role-based access. The frontend provides role-specific dashboards while the Express backend exposes REST APIs for controlled access to the MySQL database.

1.2 Problem Statement

A blood bank needs timely information about which blood groups are available, which units are nearing expiry, which hospitals have raised requests, and which requests have been fulfilled. A system that stores only a total stock count cannot accurately represent individual collection batches or their expiry dates.

RaktaSetu addresses the need for a centralized system that records donor activity, hospital requests, blood batches, donation camps, cross-match results, alerts, and administrative statistics, while ensuring that donors, hospitals, and administrators access only the operations relevant to their roles.

1.3 Scope of the Project

- Donor registration, login, profile management, donation history, nearby blood requests, and match alerts.
- Hospital registration/login, inventory search, request creation, request tracking, and request details.
- Administrator management of inventory batches, donations, camps, cross-match records, alerts, requests, reports, and dashboard statistics.
- Expiry-aware blood inventory using collection and expiry dates for every batch.
- Blood request matching and fulfillment using the oldest-expiring available batches first.
- REST APIs connecting the React frontend with the Express/MySQL backend.
- JWT authentication and role-based authorization for donor, hospital, and admin portals.

1.4 Objectives of the Project

1. Design a normalized relational database for blood-bank operations.
2. Provide separate workflows for donors, hospitals, and administrators.
3. Maintain blood stock at batch level rather than as a single aggregate number.
4. Support hospital blood requests and controlled fulfillment.
5. Record donations and update donor statistics and inventory atomically.
6. Provide alerts for critical requests and other operational conditions.
7. Expose database functionality through RESTful APIs.
8. Implement authentication and role-based authorization.
9. Provide an understandable web interface for daily blood-bank operations.

1.5 Importance of the Application

The main value of RaktaSetu is structured information management. The database maintains relationships between users, donor profiles, hospitals, requests, donations, and stock batches.

This makes it possible to retrieve operational information through SQL rather than maintaining independent manual records.

The expiry-aware inventory design is particularly relevant because each batch stores its collection date, expiry date, status, blood group, and source. The inventory view summarizes available units and near-expiry units, while fulfillment logic selects batches in expiry-date order. The application also provides a common platform for different stakeholders. Donors can see their own records, hospitals can raise and monitor requests, and administrators can manage the system and view computed statistics.

1.6 Target Users / Audience

- Donors who register with their blood group and personal details and want to track donations or nearby requests.
- Hospitals that need to search blood availability and raise or monitor requests.
- Administrators who manage inventory, requests, camps, cross-matching, alerts, and reports.
- Academic users studying relational database design, REST APIs, authentication, and full-stack DBMS applications.

CHAPTER -2 SYSTEM REQUIREMENTS

2.1 Hardware Requirements

RaktaSetu is a web application and does not require specialized client hardware. A normal laptop or desktop capable of running a modern browser, VS Code, Node.js, and MySQL is sufficient for development. A practical development configuration is a dual- or quad-core processor, 8 GB RAM or more, SSD storage, and a stable network connection.

For demonstration, the frontend and backend can run locally on the same machine. MySQL runs as the database service, the Express server listens on port 5000 by default, and the Vite development server runs the React application on its development port.

2.2 Software Requirements

- Operating system: Windows, Linux, or macOS.
- Node.js and npm for backend and frontend package management.
- MySQL 8 or a compatible MySQL server for the raktasetu database.
- Visual Studio Code or another JavaScript/SQL-capable IDE.
- Modern web browser such as Chrome, Edge, or Firefox.
- MySQL Workbench or an equivalent SQL client for executing schema.sql and seed.sql.
- Git/GitHub when version control and collaboration are required.

2.3 Development Tools and Frameworks

The frontend uses React 19, React Router, Vite, Lucide React icons, and Recharts. The backend uses Node.js with Express, mysql2 for MySQL access, bcryptjs for password hashing, jsonwebtoken for JWT authentication, cors for cross-origin requests, and dotenv for environment configuration. The database is MySQL 8 with InnoDB tables, constraints, indexes, and the v_inventory view.

The project contains separate frontend and backend packages. The backend provides development and production start commands, while the frontend provides Vite development, build, preview, and lint commands.

CHAPTER-3 TECHNOLOGY STACK

3.1 Front-End Design

The frontend is implemented using React with Vite. React components are organized into reusable UI components, layouts, authentication pages, donor pages, hospital pages, and administrator pages. React Router handles navigation between public pages and role-specific dashboard routes.

The application stores the authenticated user and JWT token in localStorage and uses a route guard in App.jsx to prevent access to donor, hospital, or admin dashboards without a valid session and matching role. API communication is centralized through the frontend API utility, which attaches the Bearer token when one is available.

- Landing page and public blood inventory view.
- Login and registration pages.
- Donor dashboard with overview, profile, history, camps, nearby requests, and alerts.
- Hospital dashboard with overview, inventory search, request creation, request list, and request details.
- Admin dashboard with overview, inventory, requests, camps, cross-match, alerts, reports, matching, and forecast views.

3.2 Back-End Design

The backend is a REST API implemented with Node.js and Express. server.js initializes CORS and JSON middleware, checks the MySQL connection, mounts route modules, provides a health endpoint, and handles unknown routes and unhandled server errors.

The route structure separates responsibilities into authentication, donors, requests, inventory, donations, camps, cross-match, alerts, admin, and metadata modules. This keeps database operations and business rules grouped by feature rather than placing all logic in a single server file.

3.3 Database Design

The database is MySQL and is created as the raktasetu database. The schema uses InnoDB tables and relational constraints. Core entities include users, donors, hospitals, blood_batches, blood_requests, donations, camps, crossmatch_records, and alerts. The v_inventory view provides a blood-group summary of currently available units, near-expiry units, and the earliest available expiry date.

Primary keys identify records, while foreign keys connect users to donor/hospital profiles, donations to donors and requests, requests to hospitals, and cross-match records to donors and requests. CHECK constraints protect values such as units and date relationships. Indexes are defined for frequently accessed fields such as blood group, status, expiry date, request status, hospital, and donor/date combinations.

3.4 Version Control

The project is structured as separate frontend and backend packages, making it suitable for Git-based version control. Source files, SQL schema, seed data, and configuration templates can be tracked while sensitive .env values should remain outside source control.

3.5 APIs or External Services

RaktaSetu primarily uses internal REST APIs between the React frontend and Express backend. The supplied project does not depend on an external blood-bank API for its core workflow. The backend communicates with MySQL through mysql2. The frontend uses the application's API layer rather than directly connecting to the database.

CHAPTER -4 SYSTEM ARCHITECTURE

4.1 High-level Architecture

```
USER
|
v
React + Vite Frontend
| HTTP/JSON + Bearer JWT
v
Express REST API
|-- Auth / Donors / Requests / Inventory
|-- Donations / Camps / Cross-match / Alerts
|-- Admin / Metadata
|
v
MySQL 8 (raktasetu)
|-- users, donors, hospitals
|-- blood_batches, blood_requests, donations
|-- camps, crossmatch_records, alerts
`-- v_inventory view
```

The architecture follows a three-layer model. The frontend is responsible for presentation and navigation, the Express backend is responsible for authentication, validation, business rules, and database access, and MySQL is responsible for persistent relational storage.

4.2 Description of Each Layer (Frontend, Backend, Database)

Frontend layer: React components render the landing page and role-specific dashboards. React Router controls navigation and route guards check the stored token and role before rendering protected dashboards.

Backend layer: Express receives HTTP requests, applies authentication/authorization middleware where required, executes SQL through the MySQL connection pool, and returns JSON responses. Transactional operations use database connections and commit/rollback logic where consistency matters.

Database layer: MySQL stores the system of record. The schema separates authentication data from donor and hospital profiles and models operational data through foreign-key relationships. The inventory view provides a reusable aggregate representation for the UI.

4.3 Deployment Architecture (Local)

The supplied project is configured primarily for local development. MySQL runs locally, the backend reads its database connection and JWT configuration from `.env`, and the Express API runs on port 5000 by default. The React application runs through Vite during development and communicates with the API through HTTP.

The project does not include a production cloud deployment configuration in the supplied source. Cloud hosting can therefore be treated as a future deployment option rather than a currently implemented feature.

CHAPTER -5 DESIGN

5.1 Database Design Overview

The schema is designed around clear entities and relationships. Authentication is centralized in users, while donors and hospitals store role-specific profile information. Operational records are kept in separate tables so that a donation, request, batch, camp, or cross-match record can be queried independently and related through keys.

The blood_batches table is central to inventory control. A row represents a collection batch with blood group, units, collection date, expiry date, status, and source information. This makes stock expiry explicit and allows the system to calculate near-expiry units without losing batch-level detail.

5.2 ER Diagram / Database Schema

USERS

- | 1----0..1 DONORS
- | 1----0..1 HOSPITALS

HOSPITALS 1----N BLOOD_REQUESTS

DONORS 1----N DONATIONS

BLOOD_REQUESTS 1----N DONATIONS (optional request link)

DONATIONS 0..1----1 BLOOD_BATCHES

BLOOD_REQUESTS 1----N CROSSMATCH_RECORDS

DONORS 0..1----N CROSSMATCH_RECORDS

BLOOD_BATCHES ----> V_INVENTORY

CAMPS and ALERTS operate as independent operational entities.

Important relationships include one user to at most one donor or hospital profile, one hospital to many blood requests, one donor to many donations, and optional links from donations and cross-match records to blood requests. Foreign-key constraints enforce these relationships.

CHAPTER -6 IMPLEMENTATION

6.1 Module-Wise Implementation

- Authentication module: registration, login, current-user lookup, password hashing, and JWT token generation.
- Donor module: profile retrieval/update, donation history, nearby requests, and match alerts.
- Hospital request module: request creation, hospital-owned request listing, request details, donor matching, and status updates.
- Inventory module: blood-group stock summary, batch listing, and administrator restocking.
- Donation module: recording donations, creating stock batches, and updating donor donation statistics.
- Camp module: public camp listing plus administrator creation and update operations.
- Cross-match module: administrator recording and viewing of compatibility records.
- Alert module: viewing alerts, raising alerts, and resolving alerts.
- Admin module: dashboard statistics, reports, demand-related analytics, and inventory summaries.
- Metadata module: public blood-group list used by the frontend.

6.2 Front-End Logic (UI Rendering and State Management)

App.jsx defines public routes and three protected dashboard routes. RequireAuth reads the saved user and token from localStorage and redirects unauthenticated users to the login page. It also compares the stored role with the route's required role.

DashboardShell and the role-specific page components organize the application into reusable navigation and content areas. The frontend communicates with the API through lib/api.js and stores authentication information through lib/auth.js. Recharts is used where dashboard analytics require graphical presentation.

6.3 API Integration

The backend exposes REST endpoints under /api. Authentication is available under /api/auth; donor operations under /api/donors; requests under /api/requests; inventory under /api/inventory; donations under /api/donations; camps under /api/camps; cross-match under /api/crossmatch; alerts under /api/alerts; administrator analytics under /api/admin; and blood-group metadata under /api/meta.

Requests and responses use JSON. Protected frontend requests send the JWT in the Authorization header using the Bearer scheme. The backend never exposes the MySQL connection directly to the browser.

6.4 Authentication and Authorisation

Passwords are stored as bcrypt hashes rather than plaintext values. On successful login the backend issues a JWT containing user identity and role information. requireAuth validates the token, while requireRole checks whether the authenticated role is permitted to access an endpoint.

Role separation is applied at both application layers. The frontend prevents navigation to dashboards belonging to other roles, while the backend independently enforces authorization. The backend check is the security boundary because client-side route guards alone cannot protect server resources.

CHAPTER 7- FEATURES

7.1 List of Main Features

- Donor and hospital registration/login with role-specific accounts.
- JWT authentication with role-based access control.
- Donor profile and donation history management.
- Hospital blood request creation and tracking.
- Blood inventory summary by all supported blood groups.
- Expiry-aware batch inventory and near-expiry visibility.
- Automatic stock creation when a donation is recorded.
- Oldest-expiring batch selection during fulfillment.
- Nearby-request and distance-based donor matching support.
- Donation camp management.
- Cross-match record management.
- Critical and operational alerts.
- Administrator dashboard statistics, reports, and forecast view.
- Responsive React dashboards for donor, hospital, and admin roles.
- Seeded demo data for development and demonstration.

7.2 How Each Feature Works

Donor workflow: a donor logs in, views profile information, checks donation history, and can see nearby hospital requests. The donor's blood group and stored latitude/longitude support matching and distance calculations.

Hospital workflow: a hospital logs in using its hospital identifier, searches inventory, raises a request containing blood group, units, urgency, patient and ward information, and tracks request status. Critical requests create an administrator alert.

Inventory workflow: administrators can inspect batch-level stock and add batches. The `v_inventory` view aggregates available units by blood group and calculates near-expiry stock. This gives the landing page and dashboards a consistent stock summary.

Donation workflow: when a donation is recorded, the backend creates a donation record and a corresponding blood batch, then updates the donor's last donation date and total donation count within a transaction.

Fulfillment workflow: when a request is fulfilled, available batches are selected in ascending expiry-date order and their unit counts are reduced. This implements expiry-aware stock dispatch and updates the request to Fulfilled when the required quantity has been supplied.

Administration workflow: administrators have access to requests, inventory, camps, cross-match records, alerts, reports, dashboard statistics, matching, and forecast pages.

CHAPTER -8 TESTING

8.1 Postman Testing (Frontend/Backend)

The backend is structured for REST API testing using tools such as Postman. Testing should be performed against the running Express server at `http://localhost:5000`. The health endpoint `/api/health` provides a simple connectivity check. Protected endpoints require an Authorization: Bearer <JWT> header.

- `POST /api/auth/register` — verify valid registration and rejection of duplicate accounts.
- `POST /api/auth/login` — verify successful login and invalid-credential handling.
- `GET /api/auth/me` — verify JWT-protected current-user retrieval.
- `GET /api/inventory` — verify blood-group inventory aggregation.
- `POST /api/requests` — verify hospital-only request creation.
- `PATCH /api/requests/:id/status` — verify authorized request status changes.
- `POST /api/donations` — verify donation creation and inventory update.
- `POST /api/crossmatch` — verify administrator-only cross-match creation.
- `PATCH /api/alerts/:id/resolve` — verify administrator alert resolution.

8.2 Integration Testing

Integration testing checks the complete path from the React UI to the Express API and then to MySQL. Representative flows include login → dashboard access, hospital request → admin visibility, donation recording → donor history and inventory update, and request fulfillment → stock reduction and request status change.

The most important database integration check is transactional consistency. Donation recording and request fulfillment contain database operations that should either complete together or roll back when an error occurs.

8.3 Tools Used for Testing

- Browser developer tools for frontend navigation, network requests, and console errors.
- Postman or an equivalent REST client for API requests and authorization headers.
- MySQL Workbench for executing SQL queries and inspecting tables and views.
- Vite development server for frontend testing.
- Node.js/Express runtime logs for backend diagnostics.

8.4 Test Cases and Results

- Valid donor login → JWT is returned and donor dashboard is accessible.
- Invalid login → request is rejected and protected dashboard access is denied.
- Donor attempting an admin endpoint → backend role middleware returns an authorization error.
- Hospital creates a positive-unit request → request is stored with Pending status.
- Critical hospital request → an administrative alert is generated.
- Donation is recorded → donation history, donor counters, and a new inventory batch are updated.
- Inventory query → all supported blood groups can be represented in the stock summary, including groups with zero stock.
- Request fulfillment → stock is consumed in expiry-date order and request fulfillment fields are updated.
- Alert resolution → alert becomes resolved and records a resolution timestamp.

These are functional test cases derived from the implemented routes and database rules. Exact execution evidence such as Postman screenshots can be inserted in the appendix after the team's final demonstration run.

CHAPTER -9 DEPLOYMENT

9.1 Steps to Deploy

1. Install MySQL and create the database by running backend/schema.sql.
2. Run backend/seed.sql once when demo data is required.
3. Copy backend/.env.example to backend/.env and provide the MySQL password and a secure JWT secret.
4. Install backend dependencies with npm install and start the API using npm run dev or npm start.
5. Install frontend dependencies with npm install in the blood-mgmt-frontend directory.
6. Start the frontend with npm run dev.
7. Open the Vite development URL and verify that the frontend can reach the backend.

9.2 Environment Configuration

The backend reads DB_HOST, DB_PORT, DB_USER, DB_PASSWORD, DB_NAME, JWT_SECRET, JWT_EXPIRES_IN, and PORT from environment variables where configured. Defaults are provided for local MySQL and port 5000, but production credentials and secrets should be supplied explicitly.

The .env file contains sensitive configuration and should not be committed to a public repository. The supplied .env.example is the appropriate template for setup.

9.3 Hosting of Frontend and Backend

The supplied project is configured primarily for local development rather than a completed cloud deployment. For a production environment, the React build can be served by a static web host or web server, while the Express API and MySQL database can run on appropriately secured servers or managed services. HTTPS, secret management, database backups, and restricted network access should be added before production use.

CHAPTER -10 CHALLENGES & LIMITATIONS

10.1 Issues Faced During Development

The main design challenge is maintaining consistency across several related entities. A hospital request affects request status, possible matches, inventory allocation, and alerts. A donation affects the donation record, donor statistics, and blood inventory. These operations require careful ordering and relational constraints.

Another challenge is representing inventory accurately. A simple blood-group counter cannot represent different expiry dates, so the system uses individual blood batches and an inventory view to derive the current summary.

10.2 Solutions Applied

- Used foreign keys and CHECK constraints to protect relational integrity.
- Used InnoDB transactions for multi-step donation and fulfillment operations.
- Used an expiry-aware `blood_batches` table instead of a single stock counter.
- Used the `v_inventory` view to centralize inventory aggregation.
- Separated backend routes by feature for easier maintenance.
- Used JWT and role middleware for server-side access control.
- Used parameterized MySQL queries for database operations.

10.3 Current Limitations or Known Gaps

- The supplied project is configured primarily for local development; a production cloud deployment is not included.
- The core workflow uses project-managed database data rather than a live external blood-bank network.
- Payment processing is not part of the blood-bank workflow.
- The application should not be treated as a medical decision system; clinical compatibility decisions require qualified personnel and validated procedures.
- Automated unit-test suites are not included in the supplied source; API and integration testing can be demonstrated manually.
- Production hardening such as managed secret storage, HTTPS termination, backups, monitoring, and rate limiting would require additional deployment work.

CHAPTER -11 FUTURE ENHANCEMENTS

11.1 Planned Features

- Production deployment with HTTPS and managed database infrastructure.
- Automated notifications for critical requests and donor matches.
- More advanced donor-hospital matching and scheduling workflows.
- Expanded analytics for demand trends and blood-group shortages.
- Automated recurring inventory and expiry alerts.
- Audit trails for sensitive administrative actions.
- Stronger automated testing for API and database workflows.

11.2 Possible Integrations or Optimisations

The current modular API structure allows future integration with messaging or notification providers, mapping/location services, and verified hospital or blood-bank networks. Such integrations should be introduced behind backend services so that provider credentials and external dependencies are not exposed to the frontend.

Database optimisation can include additional indexes based on measured query patterns, archival policies for old operational records, scheduled backups, and monitoring of slow queries. A production deployment can also introduce reverse proxies, secure secret management, and centralized logging.

CHAPTER -12 CONCLUSION

12.1 Summary of the Project

RaktaSetu is a full-stack DBMS project that models the main information flows of a blood bank through a relational MySQL database and a role-based web interface. It brings donors, hospitals, and administrators into one system and provides database-backed workflows for donations, requests, inventory, camps, cross-matching, and alerts.

The project demonstrates how database concepts become useful application features.

Relationships, constraints, indexes, views, SQL queries, and transactions directly support inventory correctness, request processing, and consistent record management.

12.2 What Was Achieved

- Implemented a MySQL schema for the major blood-bank entities.
- Implemented an Express REST API with feature-specific routes.
- Implemented JWT authentication and role-based authorization.
- Implemented React dashboards for donor, hospital, and admin roles.
- Implemented expiry-aware inventory and fulfillment logic.
- Implemented donation, request, camp, cross-match, and alert workflows.
- Implemented administrative statistics, reports, matching, and forecast views.
- Provided schema and seed scripts for reproducible local demonstration.

12.3 Skills Learned During Development

- Relational database modelling and normalization.
- SQL table creation, constraints, joins, aggregation, views, and indexing.
- Transactions and consistency in multi-step database operations.
- REST API design with Express and MySQL.
- JWT authentication and role-based authorization.
- React component design and client-side routing.
- Frontend-backend integration using JSON APIs.
- Debugging, configuration management, and project structuring.

CHAPTER -13 REFERENCES

Books

Database System Concepts — Abraham Silberschatz, Henry F. Korth, and S. Sudarshan.
Fundamentals of Database Systems — Ramez Elmasri and Shamkant B. Navathe.

Tutorials and Learning Resources

- React and React Router documentation.
- Node.js and Express documentation.
- MySQL 8 documentation and SQL reference.
- Vite documentation.
- JSON Web Token documentation.
- bcryptjs package documentation.

APIs and Tools Used

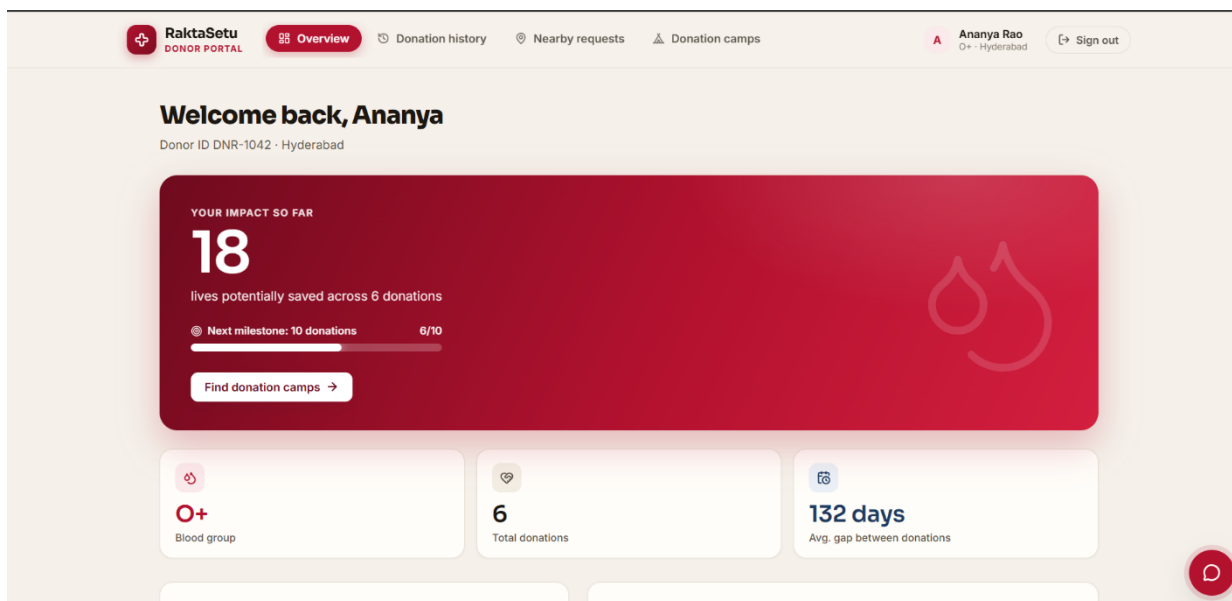
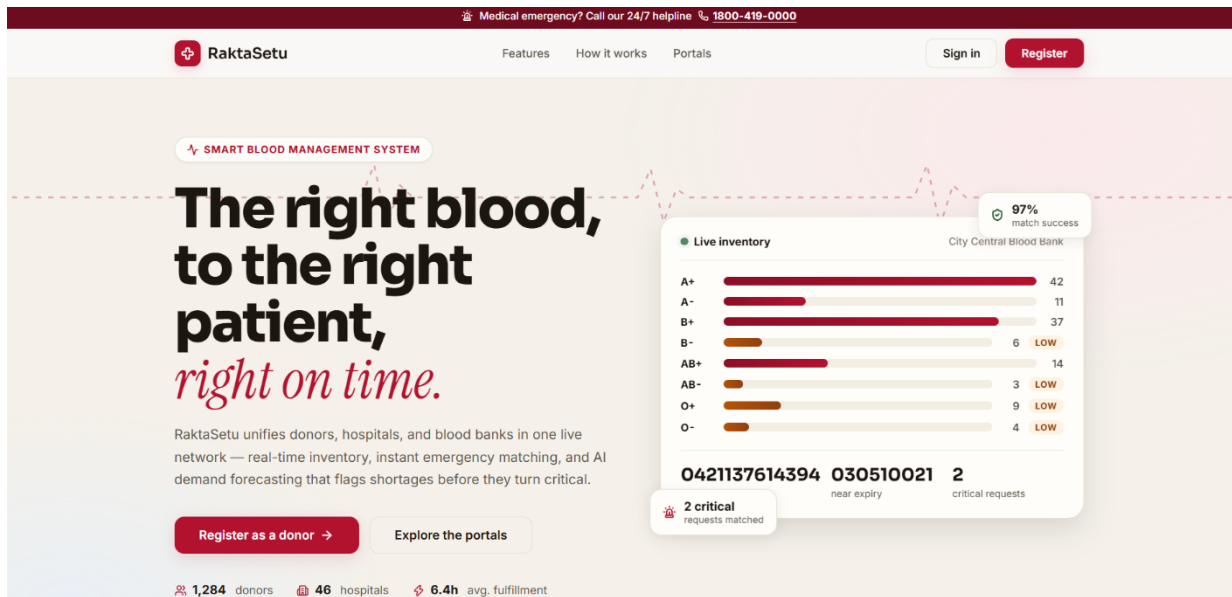
- React 19 and React DOM.
- React Router DOM.
- Vite.
- Express.
- mysql2.
- jsonwebtoken.
- bcryptjs.
- cors and dotenv.
- Lucide React.
- Recharts.
- MySQL Workbench.
- Postman or equivalent REST client for API testing.

Documentation and Research References

The primary project-specific references are the supplied RaktaSetu source files: backend/schema.sql, backend/seed.sql, backend/src/server.js, backend route modules, authentication middleware, database utility, and the React frontend source under blood-mgmt-frontend/src.

CHAPTER -14 APPENDICES

Screenshots of the app



Sample code snippets

```
// JWT role protection
router.use(requireRole("admin"));
// Expiry-aware inventory view
CREATE OR REPLACE VIEW v_inventory AS
SELECT blood_group,
       SUM(CASE WHEN status = 'available' THEN units ELSE 0 END)
AS units,
       SUM(CASE WHEN status = 'available'
                AND expiry_date <= CURDATE() + INTERVAL 3 DAY
                THEN units ELSE 0 END) AS near_expiry_units
FROM blood_batches
GROUP BY blood_group;
```

```
// Backend API health check
app.get("/api/health", (req, res) =>
  res.json({ ok: true, service: "raktasetu-backend" })
);
```

Installation / Setup Instructions

1. Open backend/schema.sql in MySQL Workbench and execute it.
2. Execute backend/seed.sql when demonstration data is required.
3. Create backend/.env from backend/.env.example and set database credentials and JWT_SECRET.
4. Run npm install in backend and start with npm run dev.
5. Run npm install in blood-mgmt-frontend and start with npm run dev.
6. Open the Vite URL and log in using a seeded demo account when applicable.

User Manual / Guide

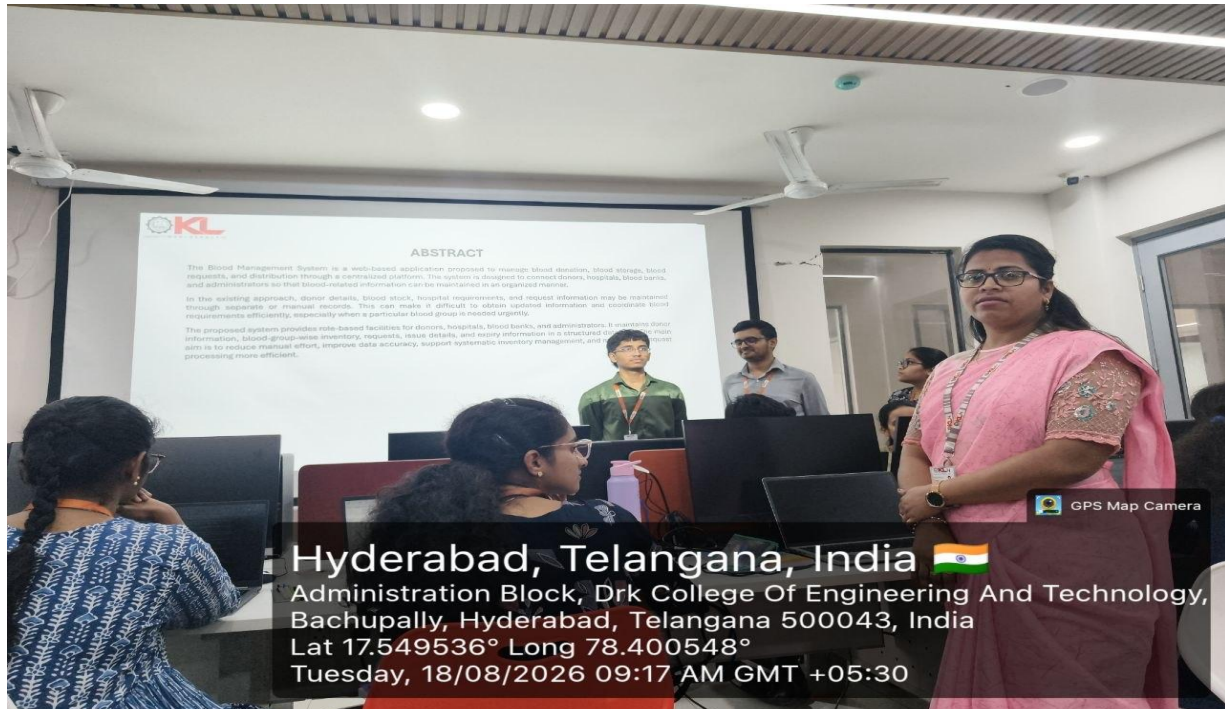
Donor: register or log in → open the donor dashboard → review profile and blood group → check donation history → view nearby requests and match alerts → use the camps section for available donation-camp information.

Hospital: log in using the hospital account → search inventory → raise a request with blood group, units, urgency, patient and ward details → monitor the request status → open request details to review matching and fulfillment information.

Admin: log in → review dashboard statistics → inspect inventory and batches → manage hospital requests → record or review donations, camps, and cross-match records → review and resolve alerts → use reports, matching, and forecast pages.

Geo Tag photos with guide





Review forms with guide signatures